

Building Components and Applications with React



Thinking in React

- ◆ When you build a user interface with React,
 - First, break it apart into pieces called *components*.
 - Then, describe the different visual states for each components.
 - Finally, connect your components together so that the data flows through them.

Start with the mockup

```
[
  { category: "Fruits", price: "$1", stocked: true, name: "Apple" },
  { category: "Fruits", price: "$1", stocked: true, name: "Dragonfruit" },
  { category: "Fruits", price: "$2", stocked: false, name: "Passionfruit" },
  { category: "Vegetables", price: "$2", stocked: true, name: "Spinach" },
  { category: "Vegetables", price: "$4", stocked: false, name: "Pumpkin" },
  { category: "Vegetables", price: "$1", stocked: true, name: "Peas" }
]
```

☐ Only show products in stock

Name	Price
------	-------

Fruits

Apple	\$1
-------	-----

Dragonfruit	\$1
-------------	-----

Passionfruit	\$2
--------------	-----

Vegetables

Spinach	\$2
---------	-----

Pumpkin	\$4
---------	-----

Peas	\$1
------	-----

Break the UI into a component hierarchy

☐ Only show products in stock

Name	Price
Fruits	
Apple	\$1
Dragonfruit	\$1
Passionfruit	\$2
Vegetables	
Spinach	\$2
Pumpkin	\$4
Peas	\$1

1. FilterableProductTable

2. SearchBar

3. ProductTable

4. ProductCategoryRow

5. ProductRow

- FilterableProductTable

- SearchBar

- ProductTable

- ProductCategoryRow

- ProductRow

Build a static version

- ◆ 5 components

- FilterableProductTable
 - SearchBar
 - ProductTable
 - ProductCategoryRow
 - ProductRow

- ◆ The components only return JSX
- ◆ The component at the top of the hierarchy (FilterableProductTable) will take the data model as a prop.

ProductRow

- FilterableProductTable
 - SearchBar
 - ProductTable
 - ProductCategoryRow
 - ProductRow

Fruits	
Apple	\$1

```
1  function ProductCategoryRow({ category }) {
2    return (
3      <tr>
4        <th colspan="2">
5          {category}
6        </th>
7      </tr>
8    );
9  }
10
11 function ProductRow({ product }) {
12   const name = product.stocked ? product.name :
13     <span style={{ color: 'red' }}>
14       {product.name}
15     </span>;
16
17   return (
18     <tr>
19       <td>{name}</td>
20       <td>{product.price}</td>
21     </tr>
22   );
23 }
```

ProductTable

- FilterableProductTable
- SearchBar
- ProductTable
 - ProductCategoryRow
 - ProductRow

Name	Price
Fruits	
Apple	\$1
Dragonfruit	\$1
Passionfruit	\$2
Vegetables	
Spinach	\$2
Pumpkin	\$4
Peas	\$1

```
25 function ProductTable({ products }) {
26   const rows = [];
27   let lastCategory = null;
28
29   products.forEach((product) => {
30     if (product.category !== lastCategory) {
31       rows.push(
32         <ProductCategoryRow
33           category={product.category}
34           key={product.category} />
35       );
36     }
37     rows.push(
38       <ProductRow
39         product={product}
40         key={product.name} />
41     );
42     lastCategory = product.category;
43   });
44
45   return (
46     <table>
47       <thead>
48         <tr>
49           <th>Name</th>
50           <th>Price</th>
51         </tr>
52       </thead>
53       <tbody>{rows}</tbody>
54     </table>
55   );
56 }
```

SearchBar

- FilterableProductTable
 - SearchBar
 - ProductTable
 - ProductCategoryRow
 - ProductRow

☐ Only show products in stock

```
58 function SearchBar() {
59   return (
60     <form>
61       <input type="text" placeholder="Search..." />
62       <label>
63         <input type="checkbox" />
64         {' '}
65         Only show products in stock
66       </label>
67     </form>
68   );
69 }
70
71 function FilterableProductTable({ products }) {
72   return (
73     <div>
74       <SearchBar />
75       <ProductTable products={products} />
76     </div>
77   );
78 }
```


Make the UI interactive

- ◆ Use *state*, **State is like a component's memory.**
- ◆ State as the *minimal* set of changing data that your app needs to remember.
- ◆ The most important principle for structuring state is to keep it **DRY** (Don't Repeat Yourself)

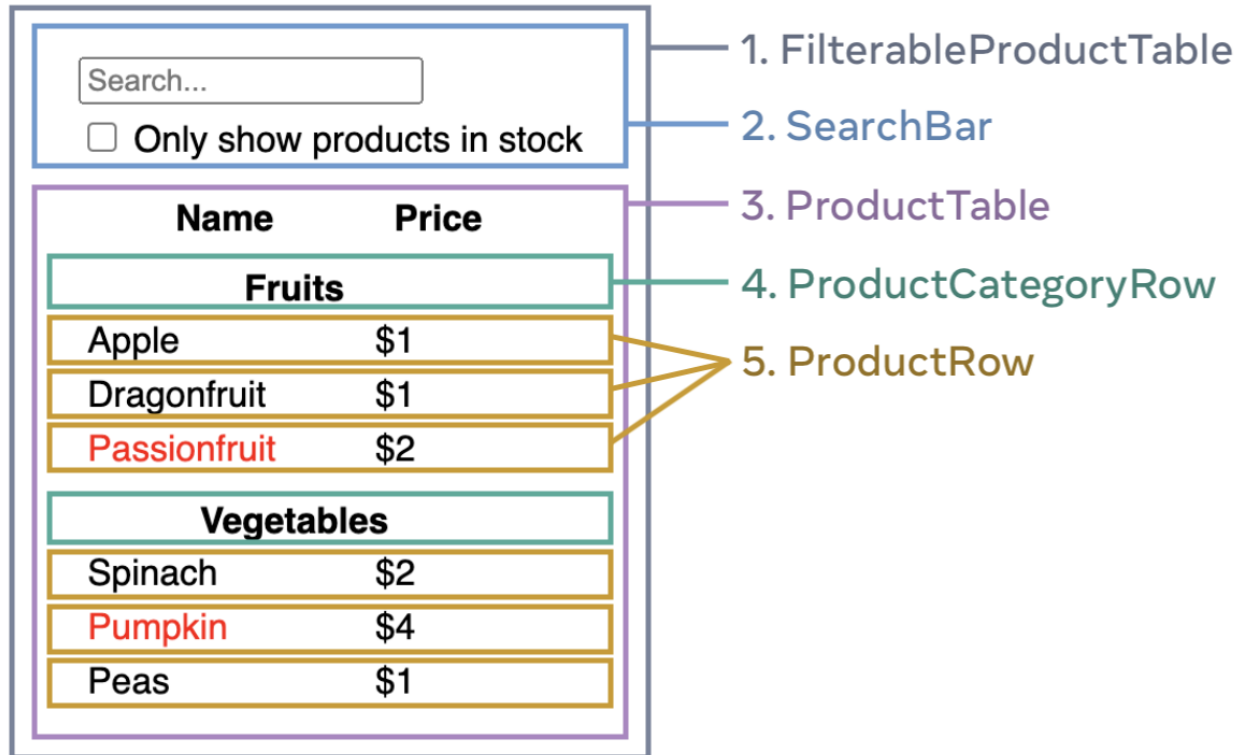
Identify the State

1. The original list of products → not state, passed in as props
2. The filtered list of products → not state, it can be computed
3. The search text the user has entered → state
4. The value of the checkbox → state, changes over time and can't be computed

Identify where the state lives

- ◆ React uses passing data from **parent to child** component
- ◆ Identify every component that renders something based on that state.
- ◆ Find their closest **common parent** component—a component above them all in the hierarchy.
 - put the state **directly** into their common parent.
 - put the state into some component **above** their common parent.

Decide where the state lives



- ◆ *SearchBar* needs state (search text and checkbox value)
- ◆ *ProductTable* needs state to filter the product list.
- ◆ *FilterableProductTable*: common parent → keep the filter text and checked state values

useState() Hook

- ◆ Add two state variables at the top of *FilterableProductTable*, and *initialize* state

```
function FilterableProductTable({ products }) {  
  const [filterText, setFilterText] = useState('');  
  const [inStockOnly, setInStockOnly] = useState(false);
```

- ◆ Pass *filterText* and *inStockOnly* to *ProductTable* and *SearchBar* as props:

```
<div>  
  <SearchBar  
    filterText={filterText}  
    inStockOnly={inStockOnly} />  
  <ProductTable  
    products={products}  
    filterText={filterText}  
    inStockOnly={inStockOnly} />  
</div>
```


Add the onChange event handlers

```
function FilterableProductTable({ products }) {  
  const [filterText, setFilterText] = useState('');  
  const [inStockOnly, setInStockOnly] = useState(false);  
  
  return (  
    <div>  
      <SearchBar  
        filterText={filterText}  
        inStockOnly={inStockOnly}  
        onFilterTextChange={setFilterText}  
        onInStockOnlyChange={setInStockOnly} />  
    </div>  
  );  
}
```

```
function SearchBar({  
  filterText,  
  inStockOnly,  
  onFilterTextChange,  
  onInStockOnlyChange  
) {  
  return (  
    <form>  
      <input  
        type="text"  
        value={filterText}  
        placeholder="Search..."  
        onChange={(e) => onFilterTextChange(e.target.value)} />  
    </form>  
  );  
}
```